МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении курсовой работы по дисциплине «Методы проектирования баз данных» на тему «Нереляционные возможности СУБД PostgreSQL»

Выполнила студентка группы
№5130201/40101 Старкова Ю.А.

Проверил доцент ВШТИИ Попов С.Г.

«____» _____ 2026 г.

Санкт-Петербург
2026

Аннотация

тут аннотация

Содержание

Введение 4

1 Постановка задачи 5

2 Представления 6
- 2.1 Создание представления 6
- 2.2 Запрос 1 8
- 2.3 Запрос 2 9

Заключение раздела «Представления» 13

3 Триггеры 14
- 3.1 Создание таблицы 14
- 3.2 Создание триггеров 17
- 3.3 Демонстрация работы триггеров 23

Заключение раздела «Триггеры» 27

4 Права доступа 28
- 4.1 Создание пользователей 28
- 4.2 Результаты работы 29

Заключение раздела «Права доступа» 33

Список литературы 34

Введение

тут введение

1 Постановка задачи

2 Представления

2.1 Создание представления

Формулировка задания на русском языке.

Создать представление, в котором необходимо для каждой коровы подсчитать количество лактаций и количество рационов, по которым она питалась.

Команда создания представления на языке SQL приведена в листинге 1.

Листинг 1: Создание представления

```
CREATE VIEW Cow_Lactation_Diet_Count AS
SELECT Cow.id_Cow, COUNT(DISTINCT Lactation.
    ↪ id_Lactation) AS lactation_count,
COUNT(DISTINCT Diet.id_Diet) AS diet_count
FROM Cow
LEFT JOIN Lactation ON Lactation.id_Cow = Cow.id_Cow
LEFT JOIN Nutrition ON Nutrition.id_Cow = Cow.id_Cow
LEFT JOIN Feeding_plan ON Feeding_plan.id_Feeding_plan
    ↪ = Nutrition.id_Feeding_plan
LEFT JOIN Diet ON Diet.id_Diet = Feeding_plan.id_Diet
GROUP BY Cow.id_Cow;
```

На рисунке 1 продемонстрирована часть таблицы — результат выполнения команды создания представления.

id_cow	lactation_count	diet_count
1	30	16
2	25	24
3	30	16
4	30	21
5	35	18
6	25	19
7	15	15
8	35	15
9	35	22
10	10	20
11	20	22
12	20	23
13	15	17
14	35	20
15	30	19

Рис. 1: Результат выполнения команды создания представления

На рисунке 2 продемонстрирован план выполнения SQL-запроса, лежащего в основе созданного представления.

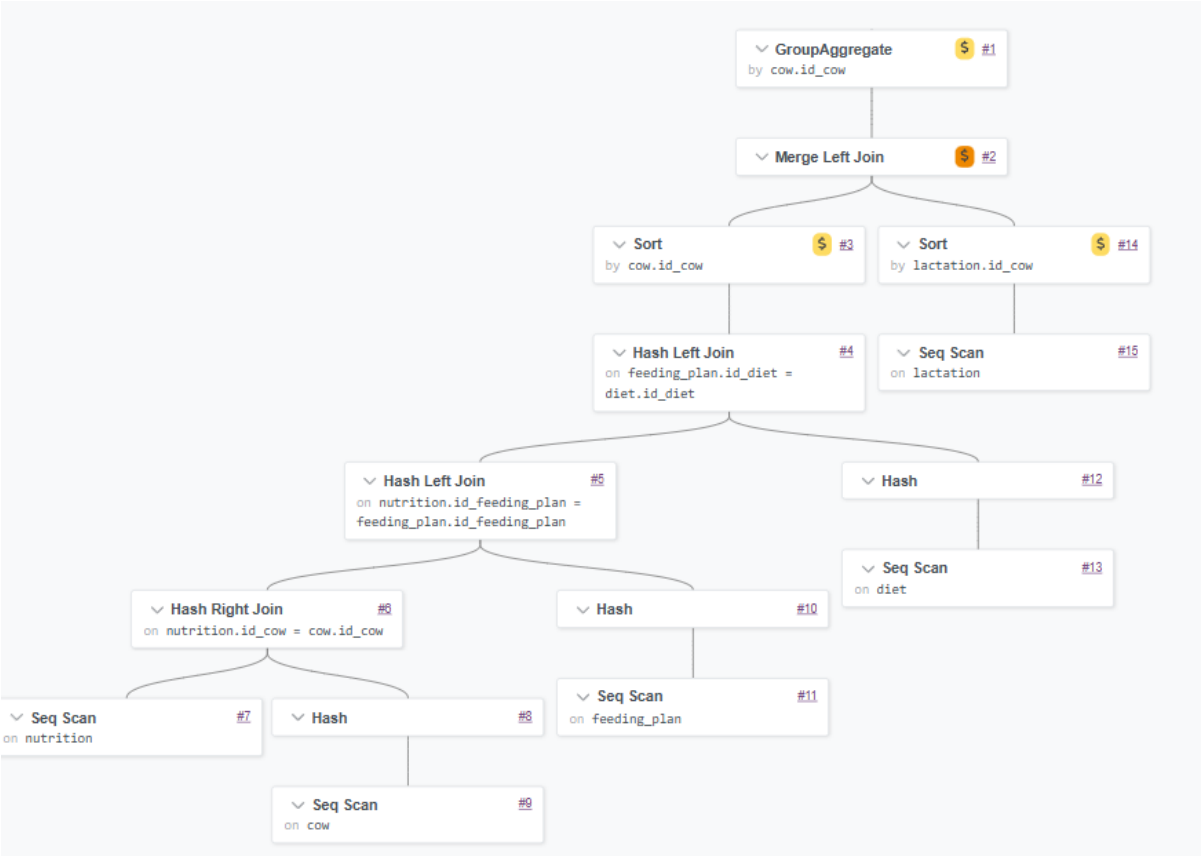


Рис. 2: Графическое представление последовательности выполнения запроса при создании представления

Текстовое объяснение последовательности выполнения запроса, лежащего в основе создания представления.

В запросе сначала осуществляется обращение к базовой таблице `Cow`. Затем к ней последовательно присоединяются таблицы `Lactation`, `Nutrition`, `Feeding_plan` и `Diet` с использованием оператора левого внешнего соединения (`LEFT JOIN`). После объединения данных строки группируются по уникальному идентификатору коровы (`Cow.id_Cow`) с помощью `GROUP BY`. Для каждой группы (каждой коровы) с помощью агрегатной функции `COUNT` и ключевого слова `DISTINCT` вычисляется количество уникальных записей лактаций и уникальных рационов. Итоговый результат сохраняется в базе данных как виртуальная таблица (представление) под именем `Cow_Lactation_Diet_Count`.

Реляционная формула команды.

$$\text{Cow_Lactation_Diet_Count} \leftarrow \mathcal{G}_{\substack{\text{Cow.id_Cow},\\ \text{COUNT(DISTINCT Lactation.id_Lactation)}\rightarrow\text{lactation_count},\\ \text{COUNT(DISTINCT Diet.id_Diet)}\rightarrow\text{diet_count}}} \Big(\text{Cow} \overset{\text{LEFT}}{\bowtie}_{\text{Cow.id_Cow=Lactation.id_Cow}} \text{Lactation} \overset{\text{LEFT}}{\bowtie}_{\text{Cow.id_Cow=Nutrition.id_Cow}} \text{Nutrition} \overset{\text{LEFT}}{\bowtie}_{\text{Nutrition.id_Feeding_plan=Feeding_plan.id_Feeding_plan}} \text{Feeding_plan} \overset{\text{LEFT}}{\bowtie}_{\text{Feeding_plan.id_Diet=Diet.id_Diet}} \text{Diet} \Big)$$

2.2 Запрос 1

Формулировка запроса 1 на русском языке.
Подсчитать количество коров, у которых было больше 2 лактаций.
Код запроса на языке SQL представлен в листинге 2.

Листинг 2: Запрос 1

```
1  SELECT COUNT(*) FROM Cow_Lactation_Diet_Count
     ↪ WHERE lactation_count > 2;
```

На рисунке 3 продемонстрирован результат выполнения запроса.

```
cows_db=> select count(*) from Cow_Lactation_Diet_Count where lactation_count > 2;
 count
-------
  6574
(1 row)
```

Рис. 3: Результат выполнения запроса 1

На рисунке 4 продемонстрировано графическое представление последовательности выполнения запроса 1.

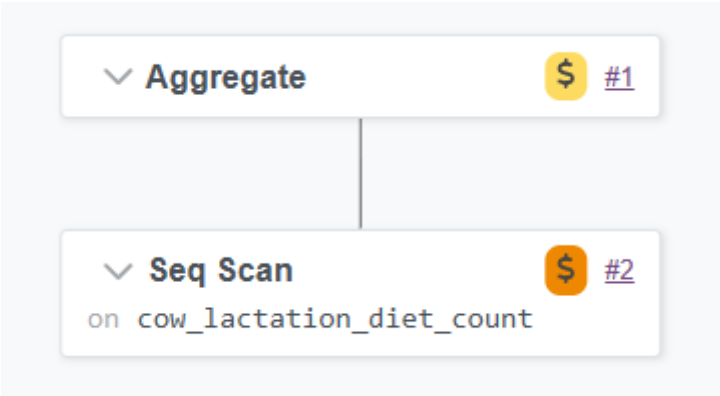


Рис. 4: Графическое представление последовательности выполнения запроса 1

Текстовое объяснение последовательности выполнения запроса 1.

В запросе осуществляется обращение к ранее созданному представлению
`Cow_Lactation_Diet_Count`. Сначала с помощью оператора `WHERE` отфильтровываются записи, оставляя только те строки, в которых значение атрибута `lactation_count` (количество лактаций) строго больше двух. Затем для полученного набора данных применяется агрегатная функция `COUNT`, которая подсчитывает итоговое количество строк (коров), удовлетворяющих заданному условию.

Реляционная формула запроса.

$$\mathcal{G}_{\text{COUNT}(*)\to\text{cow_count}}(\sigma_{\text{lactation_count}>2}(\text{Cow_Lactation_Diet_Count}))$$

2.3 Запрос 2

Формулировка запроса 2 на русском языке.

Для каждой коровы вывести возраст, количество лактаций, количество рационов и кормов по которым она питалась.

Код запроса на языке SQL представлен в листинге 3.

Листинг 3: Запрос 2

```
1   SELECT Cow.id_Cow, Cow.Age,
      ↪ Cow_Lactation_Diet_Count.lactation_count,
```

```
    Cow_Lactation_Diet_Count.diet_count, COUNT(
        ↪ DISTINCT Feeds.id_Feeds) AS feed_count
FROM Cow_Lactation_Diet_Count
JOIN Cow ON Cow.id_Cow = Cow_Lactation_Diet_Count.
    ↪ id_Cow
LEFT JOIN Nutrition ON Nutrition.id_Cow = Cow.
    ↪ id_Cow
LEFT JOIN Feeding_plan ON Feeding_plan.
    ↪ id_Feeding_plan = Nutrition.id_Feeding_plan
LEFT JOIN Feeds ON Feeds.id_feeds = Feeding_plan.
    ↪ id_feeds
GROUP BY Cow.id_Cow, Cow.Age,
    ↪ Cow_Lactation_Diet_Count.lactation_count,
    ↪ Cow_Lactation_Diet_Count.diet_count;
```

На рисунке 5 продемонстрирован результат выполнения запроса (часть таблицы).

id_cow	age	lactation_count	diet_count	feed_count
1	6	30	16	16
2	1	25	24	25
3	13	30	16	13
4	13	30	21	17
5	11	35	18	18
6	13	25	19	17
7	4	15	15	15
8	5	35	15	16
9	10	35	22	21
10	1	10	20	18
11	11	20	22	18
12	6	20	23	17
13	15	15	17	15
14	13	35	20	19
15	13	30	19	16

Рис. 5: Результат выполнения запроса 2

На рисунке 6 продемонстрировано графическое представление последовательности выполнения запроса 2.

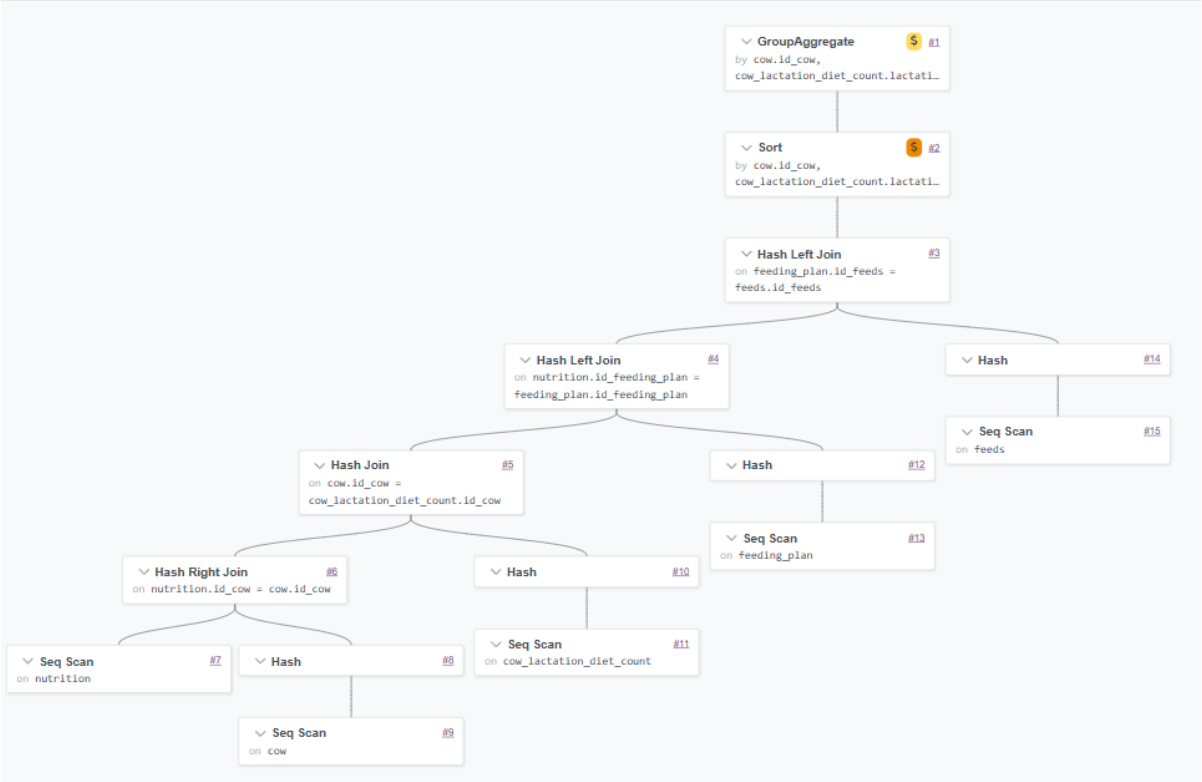


Рис. 6: Графическое представление последовательности выполнения запроса 2

Текстовое объяснение последовательности выполнения запроса 2.

В запросе осуществляется обращение к представлению `Cow_Lactation_Diet_Count`, к которому с помощью внутреннего соединения (`JOIN`) присоединяется базовая таблица `Cow`. Затем к полученному результату последовательно присоединяются таблицы `Nutrition`, `Feeding_plan` и `Feeds` с использованием оператора левого внешнего соединения (`LEFT JOIN`). После объединения данных строки группируются по идентификатору и возрасту коровы, а также по количествам лактаций и рационов, полученным из представления. В завершение для каждой группы с помощью агрегатной функции `COUNT` и ключевого слова `DISTINCT` вычисляется количество уникальных идентификаторов кормов (`feed_count`).

Реляционная формула запроса.

$$\mathcal{G}_{\substack{\text{Cow.id_Cow, Cow.Age,}\\ \text{Cow_Lactation_Diet_Count.lactation_count,}\\ \text{Cow_Lactation_Diet_Count.diet_count,}\\ \text{COUNT(DISTINCT Feeds.id_Feeds)}\rightarrow\text{feed_count}}}\Big($$

$$\text{Cow_Lactation_Diet_Count} \bowtie_{\text{Cow_Lactation_Diet_Count.id_cow=Cow.id_cow}} \text{Cow}$$

$$\overset{\text{LEFT}}{\bowtie}_{\text{Cow.id_cow=Nutrition.id_cow}} \text{Nutrition}$$

$$\overset{\text{LEFT}}{\bowtie}_{\text{Nutrition.id_Feeding_plan=Feeding_plan.id_Feeding_plan}} \text{Feeding_plan}$$

$$\overset{\text{LEFT}}{\bowtie}_{\text{Feeding_plan.id_feeds=Feeds.id_feeds}} \text{Feeds)}$$

Заключение раздела «Представления»

В результате работы было создано представление `Cow_lactation_diet_count`, в котором для каждой коровы подсчитаны количество лактаций и рационов, по которым она питалась. Были составлены два запроса: первый — непосредственно к представлению, второй — с использованием представления при объединении с другими таблицами. К каждому запросу составлены реляционные формулы и представлены графические планы выполнения.

3 Триггеры

Формулировка задания.

Создать таблицу, в которой с помощью триггеров для каждой коровы поддерживаются актуальные значения количества лактаций и назначенных рационов питания.

3.1 Создание таблицы

Текстовое объяснение последовательности выполнения команды создания таблицы.

При выполнении команды создается физическая таблица `Cow_lactation_diet_count`. В структуре таблицы определяются три поля целочисленного типа (`INT`): первичный ключ `id_Cow` для идентификации коровы, а также столбцы `Lactation_count` и `Diet_count`, предназначенные для хранения актуальных значений количества лактаций и рационов каждой коровы.

SQL-код создания таблицы `Cow_lactation_diet_count` приведен в листинге 4.

Листинг 4: Создание таблицы Cow_lactation_diet_count

```
1 CREATE TABLE IF NOT EXISTS Cow_lactation_diet_count (
2     id_Cow INT PRIMARY KEY,
3     Lactation_count INT,
4     Diet_count INT
5 );
```

Текстовое объяснение последовательности выполнения команды заполнения таблицы.

Для заполнения таблицы `Cow_lactation_diet_count` используется команда `INSERT INTO` в сочетании с подзапросом, считывающим выборку `SELECT`. В подзапросе к таблице `Cow` последовательно присоединяются таблицы `Lactation, Nutrition, Feeding_plan` и `Diet` с помощью левого внешнего соединения (`LEFT JOIN`). Данные группируются по ключевому полю `Cow.id_Cow`, после чего агрегатная функция `COUNT` с ключевым словом `DISTINCT` рассчитывает количество уникальных лактаций и рационов для каждой коровы. Полученные значения вставляются в соответствующие столбцы созданной таблицы.

В листинге 5 приведен SQL-код заполнения таблицы `Cow_lactation_diet_count` данными.

Листинг 5: Заполнение таблицы Cow_lactation_diet_count

```
INSERT INTO Cow_lactation_diet_count (id_Cow,
    ↪ Lactation_count, Diet_count)
SELECT Cow.id_Cow,
    COUNT(DISTINCT Lactation.id_Lactation),
    COUNT(DISTINCT Diet.id_Diet)
FROM Cow
LEFT JOIN Lactation ON Lactation.id_Cow = Cow.id_Cow
LEFT JOIN Nutrition ON Nutrition.id_Cow = Cow.id_Cow
LEFT JOIN Feeding_plan ON Feeding_plan.id_Feeding_plan
    ↪ = Nutrition.id_Feeding_plan
LEFT JOIN Diet ON Diet.id_Diet = Feeding_plan.id_Diet
GROUP BY Cow.id_Cow;
```

На рисунке 7 представлена схема базы данных на английском языке с добавлением созданной таблицы `Cow_lactation_diet_count` и указанием триггеров.

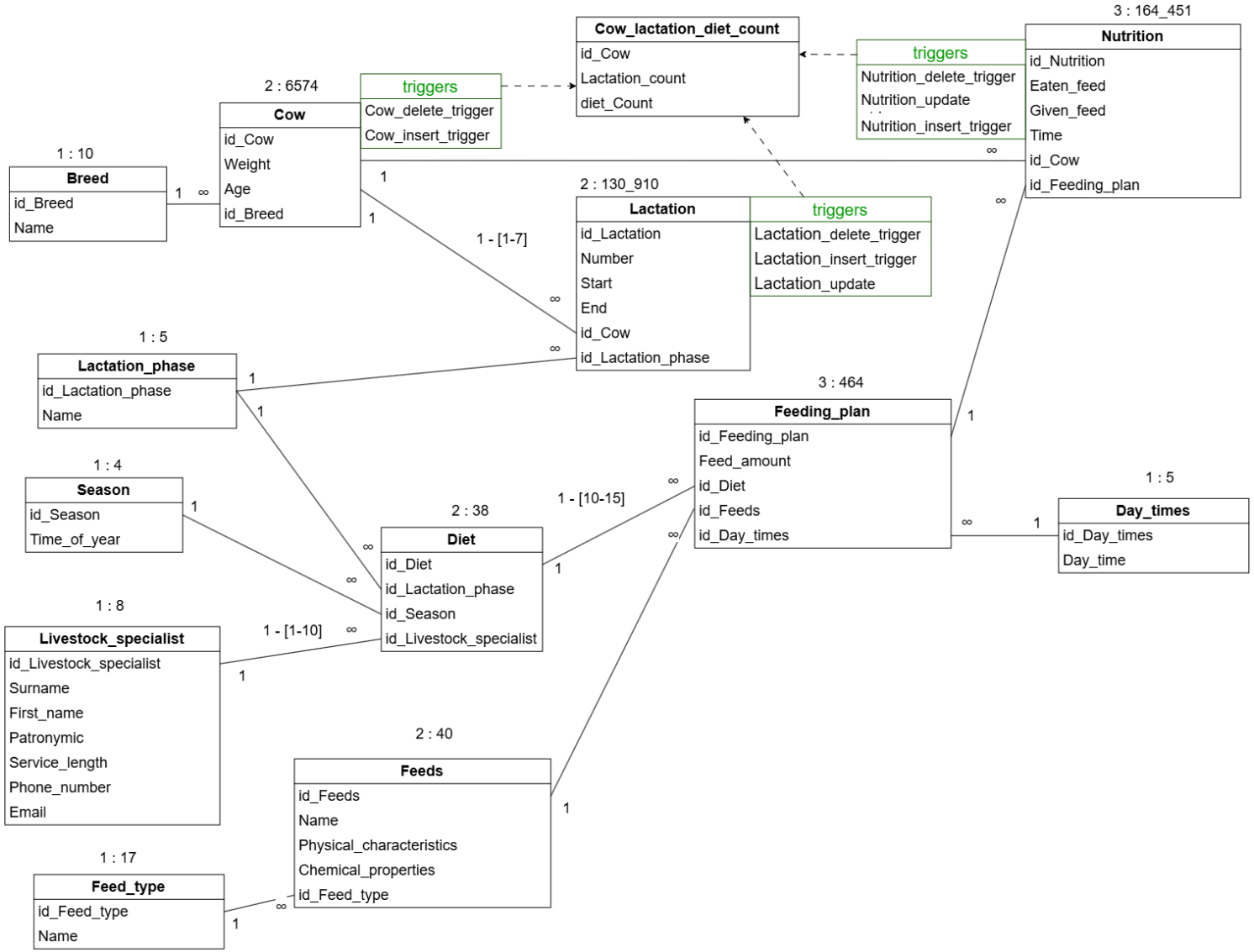


Рис. 7: Схема базы данных на английском языке с триггерами

3.2 Создание триггеров

В листинге 6 приведена SQL-команда создания триггера, срабатывающего после добавления новой записи в таблицу `Cow`.

Листинг 6: Создание триггера Cow_insert_trigger

```
CREATE TRIGGER Cow_insert_trigger
    AFTER INSERT ON Cow
    FOR EACH ROW EXECUTE FUNCTION
        ↪ Cow_insert_trigger_function();
```

В листинге 7 представлена SQL-команда создания функции `Cow_insert_trigger_function`. Функция вызывается триггером `Cow_insert_trigger` и добавляет в таблицу `Cow_lactation_diet_count` новую запись для зарегистрированной коровы, инициализируя начальные значения количества лактаций и рационов нулями.

Листинг 7: Создание функции Cow_insert_trigger_function

```
CREATE OR REPLACE FUNCTION Cow_insert_trigger_function
    ↪ ()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Cow_lactation_diet_count (id_Cow,
        ↪ Lactation_count, Diet_count)
    VALUES (NEW.id_Cow, 0, 0);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

В листинге 8 представлена SQL-команда создания триггера, срабатывающего после удаления записи из таблицы `Cow`.

Листинг 8: Создание триггера Cow_delete_trigger

```
CREATE TRIGGER Cow_delete_trigger
    AFTER DELETE ON Cow
    FOR EACH ROW EXECUTE FUNCTION
        ↪ Cow_delete_trigger_function();
```

В листинге 9 приведена SQL-команда создания функции `Cow_delete_trigger_function`. Функция вызывается триггером `Cow_delete_trigger` и удаляет соответствующую запись из таблицы `Cow_lactation_diet_count` по идентификатору удаляемой коровы (`OLD.id_Cow`).

Листинг 9: Создание функции Cow_delete_trigger_function

```
CREATE OR REPLACE FUNCTION Cow_delete_trigger_function
    ↪ ()
RETURNS TRIGGER AS $$
BEGIN
    DELETE FROM Cow_lactation_diet_count WHERE id_Cow
        ↪ = OLD.id_Cow;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

В листинге 10 представлена SQL-команда создания триггера, срабатывающего после добавления новой записи в таблицу `Nutrition`.

Листинг 10: Создание триггера Nutrition_insert_trigger

```
CREATE TRIGGER Nutrition_insert_trigger
    AFTER INSERT ON Nutrition
    FOR EACH ROW EXECUTE FUNCTION
        ↪ Nutrition_insert_trigger_function();
```

В листинге 11 приведена SQL-команда создания функции `Nutrition_insert_trigger_function`. Функция вызывается триггером `Nutrition_insert_trigger` и выполняет перерасчет количества уникальных рационов для коровы `NEW.id_Cow` (обращение к значению поля id_Cow из строки, которая добавлена в таблицу `Nutrition`) путем объединения таблиц `Nutrition, Feeding_plan` и `Diet` с помощью `LEFT JOIN`. В результате работы функции обновляется значение поля `diet_count` в таблице `Cow_lactation_diet_count`.

Листинг 11: Создание функции Nutrition_insert_trigger_function

```
CREATE OR REPLACE FUNCTION
    ↪ Nutrition_insert_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE Cow_lactation_diet_count
    SET diet_count = (
        SELECT COUNT(DISTINCT Diet.id_Diet)
        FROM Nutrition
        LEFT JOIN Feeding_plan ON Feeding_plan.
            ↪ id_Feeding_plan = Nutrition.
            ↪ id_Feeding_plan
```

```
9           LEFT JOIN Diet ON Diet.id_Diet = Feeding_plan.
                ↪ id_Diet
10          WHERE Nutrition.id_Cow = NEW.id_Cow
11      )
12      WHERE id_Cow = NEW.id_Cow;
13      RETURN NEW;
14  END;
15  $$ LANGUAGE plpgsql;
```

В листинге 12 представлена SQL-команда создания триггера, срабатывающего после удаления записи из таблицы `Nutrition`.

Листинг 12: Создание триггера Nutrition_delete_trigger

```
1  CREATE TRIGGER Nutrition_delete_trigger
2      AFTER DELETE ON Nutrition
3      FOR EACH ROW EXECUTE FUNCTION
           ↪ Nutrition_delete_trigger_function();
```

В листинге 13 приведена SQL-команда создания функции `Nutrition_delete_trigger_function`. Функция вызывается триггером `Nutrition_delete_trigger` и выполняет перерасчет количества уникальных рационов для коровы `OLD.id_Cow` (обращение к значению поля id_Cow из строки, которая удалена из таблицы `Nutrition`) после удаления соответствующей записи. Результат повторного вычисления обновляет значение столбца `diet_count` в таблице `Cow_lactation_diet_count`.

Листинг 13: Создание функции Nutrition_delete_trigger_function

```
1  CREATE OR REPLACE FUNCTION
       ↪ Nutrition_delete_trigger_function()
2  RETURNS TRIGGER AS $$
3  BEGIN
4      UPDATE Cow_lactation_diet_count
5      SET diet_count = (
6          SELECT COUNT(DISTINCT Diet.id_Diet)
7          FROM Nutrition
8          LEFT JOIN Feeding_plan ON Feeding_plan.
               ↪ id_Feeding_plan = Nutrition.
               ↪ id_Feeding_plan
9          LEFT JOIN Diet ON Diet.id_Diet = Feeding_plan.
               ↪ id_Diet
10         WHERE Nutrition.id_Cow = OLD.id_Cow
```

```
    )
    WHERE id_Cow = OLD.id_Cow;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

В листинге 14 представлена SQL-команда создания триггера, срабатывающего после обновления записи в таблице `Nutrition`. Данная реализация рассчитана на случай изменения только плана кормления.

Листинг 14: Создание триггера Nutrition_update_trigger

```
CREATE TRIGGER Nutrition_update_trigger
    AFTER UPDATE ON Nutrition
    FOR EACH ROW EXECUTE FUNCTION
        ↪ Nutrition_update_trigger_function();
```

В листинге 15 приведена SQL-команда создания функции `Nutrition_update_trigger_function`. Функция вызывается триггером `Nutrition_update_trigger` после изменения записи в таблице `Nutrition` и выполняет перерасчет количества уникальных рационов для коровы `NEW.id_Cow`, заменяя устаревшее значение в столбце `diet_count` таблицы `Cow_lactation_diet_count` на актуальное.

Листинг 15: Создание функции Nutrition_update_trigger_function

```
CREATE OR REPLACE FUNCTION
    ↪ Nutrition_update_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE Cow_lactation_diet_count
    SET diet_count = (
        SELECT COUNT(DISTINCT Diet.id_Diet)
        FROM Nutrition
        LEFT JOIN Feeding_plan ON Feeding_plan.
            ↪ id_Feeding_plan = Nutrition.
            ↪ id_Feeding_plan
        LEFT JOIN Diet ON Diet.id_Diet = Feeding_plan.
            ↪ id_Diet
        WHERE Nutrition.id_Cow = NEW.id_Cow
    )
    WHERE id_Cow = NEW.id_Cow;
    RETURN NEW;
```

```
14 END;
15 $$ LANGUAGE plpgsql;
```

В листинге 16 представлена SQL-команда создания триггера, срабатывающего после добавления новой записи в таблицу `Lactation`.

Листинг 16: Создание триггера Lactation_insert_trigger

```
1 CREATE TRIGGER Lactation_insert_trigger
2     AFTER INSERT ON Lactation
3     FOR EACH ROW EXECUTE FUNCTION
        ↪ Lactation_insert_trigger_function();
```

В листинге 17 приведена SQL-команда создания функции `Lactation_insert_trigger_function`. Функция вызывается триггером `Lactation_insert_trigger` при добавлении нового периода лактации и выполняет перерасчет общего количества лактаций для соответствующей коровы `NEW.id_Cow`, обновляя значение столбца `lactation_count` в таблице `Cow_lactation_diet_count`.

Листинг 17: Создание функции Lactation_insert_trigger_function

```
1  CREATE OR REPLACE FUNCTION
     ↪ Lactation_insert_trigger_function()
2  RETURNS TRIGGER AS $$
3  BEGIN
4      UPDATE Cow_lactation_diet_count
5      SET lactation_count =
6          (SELECT COUNT(*) FROM Lactation WHERE id_Cow =
             ↪ NEW.id_Cow)
7      WHERE id_Cow = NEW.id_Cow;
8      RETURN NEW;
9  END;
10 $$ LANGUAGE plpgsql;
```

В листинге 18 представлена SQL-команда создания триггера, срабатывающего после удаления записи из таблицы `Lactation`.

Листинг 18: Создание триггера Lactation_delete_trigger

```
1 CREATE TRIGGER Lactation_delete_trigger
2     AFTER DELETE ON Lactation
3     FOR EACH ROW EXECUTE FUNCTION
        ↪ Lactation_delete_trigger_function();
```

В листинге 19 приведена SQL-команда создания функции `Lactation_delete_trigger_function`. Функция вызывается триггером `Lactation_delete_trigger` и выполняет перерасчет количества лактаций для соответствующей коровы `OLD.id_Cow`, обновляя значение столбца `lactation_count` таблицы `Cow_lactation_diet_count`.

Листинг 19: Создание функции Lactation_delete_trigger_function

```
CREATE OR REPLACE FUNCTION
    ↪ Lactation_delete_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE Cow_lactation_diet_count
    SET lactation_count =
        (SELECT COUNT(*) FROM Lactation WHERE id_Cow =
            ↪ OLD.id_Cow)
    WHERE id_Cow = OLD.id_Cow;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

В листинге 20 представлена SQL-команда создания триггера, срабатывающего после обновления записи в таблице `Lactation`. Данная реализация не рассчитана на случай изменения `id_Cow` коровы.

Листинг 20: Создание триггера Lactation_update_trigger

```
CREATE TRIGGER Lactation_update_trigger
    AFTER UPDATE ON Lactation
    FOR EACH ROW EXECUTE FUNCTION
        ↪ Lactation_update_trigger_function();
```

В листинге 21 приведена SQL-команда создания функции `Lactation_update_trigger_function`. Функция вызывается триггером `Lactation_update_trigger` и выполняет перерасчет количества лактаций для соответствующей коровы `NEW.id_Cow`, обновляя значение столбца `lactation_count` в таблице `Cow_lactation_diet_count`.

Листинг 21: Создание функции Lactation_update_trigger_function

```
CREATE OR REPLACE FUNCTION
    ↪ Lactation_update_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
```

```
    UPDATE Cow_lactation_diet_count
    SET lactation_count =
        (SELECT COUNT(*) FROM Lactation WHERE id_Cow =
            ↪ NEW.id_Cow)
    WHERE id_Cow = NEW.id_Cow;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

3.3 Демонстрация работы триггеров

В таблице 1 представлено описание изменений таблицы `Cow_lactation_diet_count` посредством триггеров в зависимости от изменения данных в таблицах `Cow`, `Nutrition`, `Lactation`.

Таблица 1: Описание действий триггеров

№	Действие
1	Добавление коровы в таблицу `Cow`. В таблице `Cow_lactation_diet_count` создаётся запись с нулевыми значениями количества лактаций и рационов.
2	Удаление коровы из таблицы `Cow`. Из таблицы `Cow_lactation_diet_count` удаляется соответствующая запись.
3	Добавление записи питания в таблицу `Nutrition`. Пересчитывается количество уникальных рационов для указанной коровы в таблице `Cow_lactation_diet_count`.
4	Удаление записи питания из таблицы `Nutrition`. Пересчитывается количество уникальных рационов для указанной коровы в таблице `Cow_lactation_diet_count`.
5	Изменение плана кормления в таблице `Nutrition`. Пересчитывается количество уникальных рационов для указанной коровы в таблице `Cow_lactation_diet_count`.
6	Добавление лактации в таблицу `Lactation`. Пересчитывается общее количество лактаций для указанной коровы в таблице `Cow_lactation_diet_count`.
7	Удаление лактации из таблицы `Lactation`. Пересчитывается общее количество лактаций для указанной коровы в таблице `Cow_lactation_diet_count`.

На рисунке 8 представлено появление новой записи в таблице `Cow_lactation_diet_count` после добавления записи в таблицу `Cow`.

```
cows_db=> SELECT COUNT(*) FROM cow_lactation_diet_count;
 count
-------
  6575
(1 row)

cows_db=> INSERT INTO Cow (Weight, Age, id_Breed) VALUES (167, 1, 3);
INSERT 0 1
cows_db=> SELECT * FROM Cow WHERE Weight=167 AND Age=1;
 id_cow | weight | age | id_breed
--------+--------+-----+----------
   6578 |    167 |   1 |        3
(1 row)

cows_db=> SELECT * FROM Cow_lactation_diet_count WHERE id_Cow=6578;
 id_cow | lactation_count | diet_count
--------+-----------------+------------
   6578 |               0 |          0
(1 row)

cows_db=>
```

Рис. 8: Результат добавления новой записи в таблицу Cow

На рисунке 9 представлены изменения данных в таблице `Cow_lactation_diet_count`: отсутствие данных о корове, запись которой была удалена из таблицы `Cow`.

```
cows_db=> DELETE FROM Cow WHERE id_Cow=6578;
DELETE 1
cows_db=> SELECT COUNT(*) FROM cow_lactation_diet_count;
 count
-------
  6575
(1 row)

cows_db=> SELECT * FROM Cow_lactation_diet_count WHERE id_Cow=6578;
 id_cow | lactation_count | diet_count
--------+-----------------+------------
(0 rows)

cows_db=>
```

Рис. 9: Результат удаления записи из таблицы Cow

На рисунке 10 продемонстрировано уменьшение количества лакта-

ций в таблице Cow_lactation_diet_count после удаления записи в таблице `Lactation`.

```
cows_db=> select * from lactation where id_cow=6070;
 id_lactation | number |   start    |    End     | id_cow | id_lactation_phase
--------------+--------+------------+------------+--------+--------------------
       120776 |      1 | 2022-12-03 | 2022-12-27 |   6070 |                  1
       120777 |      1 | 2022-12-27 | 2023-05-18 |   6070 |                  2
       120778 |      1 | 2023-05-18 | 2023-06-18 |   6070 |                  3
       120779 |      1 | 2023-06-18 | 2023-07-16 |   6070 |                  4
       120780 |      1 | 2023-07-16 | 2023-08-24 |   6070 |                  5
(5 rows)

cows_db=> DELETE FROM Lactation WHERE id_Cow=6070 AND id_lactation_phase=1;
DELETE 1
cows_db=> SELECT * FROM Cow_lactation_diet_count WHERE id_cow=6070;
 id_cow | lactation_count | diet_count
--------+-----------------+------------
   6070 |               4 |         12
(1 row)

cows_db=>
```

Рис. 10: Результат удаления записи из таблицы Lactation

На рисунке 11 продемонстрировано уменьшение количества рационов в таблице `Cow_lactation_diet_count` после изменения плана кормления в таблице `Nutrition`.

```
cows_db=> SELECT * FROM Nutrition WHERE id_cow=6070;
 id_nutrition | id_cow | id_feeding_plan
--------------+--------+-----------------
       151816 |   6070 |             138
       151817 |   6070 |              47
       151818 |   6070 |              65
       151819 |   6070 |             134
       151820 |   6070 |             397
       151821 |   6070 |             449
       151822 |   6070 |             345
       151823 |   6070 |             350
       151824 |   6070 |             415
       151825 |   6070 |             359
       151826 |   6070 |             364
       151827 |   6070 |             367
       151828 |   6070 |             168
       151829 |   6070 |             343
       151830 |   6070 |              41
       151831 |   6070 |             443
       151832 |   6070 |             402
       151833 |   6070 |             230
       151834 |   6070 |             390
       151835 |   6070 |             167
(20 rows)

cows_db=> SELECT * FROM Cow_lactation_diet_count WHERE id_cow=6070;
 id_cow | lactation_count | diet_count
--------+-----------------+------------
   6070 |               5 |         11
(1 row)

cows_db=> UPDATE Nutrition SET id_feeding_plan=192 WHERE id_Nutrition=151816;
UPDATE 1
cows_db=> SELECT * FROM Cow_lactation_diet_count WHERE id_cow=6070;
 id_cow | lactation_count | diet_count
--------+-----------------+------------
   6070 |               5 |         12
(1 row)

cows_db=>
```

Рис. 11: Изменение данных при обновлении плана кормления в таблице Nutrition

Заключение раздела «Триггеры»

В результате работы была создана таблица `Cow_lactation_diet_count` и заполнена данными на основе агрегатных вычислений по таблицам `Cow`, `Lactation`, `Nutrition`, `Feeding_plan` и `Diet`. Для автоматического поддержания актуальности данных созданы 7 триггеров: для таблицы `Cow` — при добавлении и удалении записей; для таблицы `Nutrition` — при добавлении, удалении и изменении плана кормления; для таблицы `Lactation` — при добавлении и удалении периодов лактации.

4 Права доступа

Формулировка задания. Создать двух пользователей с следующими правами: один из пользователей должен иметь право только на просмотр таблицы `Cow_lactation_diet_count`, а второй пользователь — на просмотр и изменение таблиц `Cow`, `Lactation`, `Nutrition`.

4.1 Создание пользователей

В ходе работы были созданы два пользователя: resultsReader, resultsEditor (см. листинг 22).

Листинг 22: Создание пользователей

```
1 CREATE USER resultsReader WITH PASSWORD '*';
2 CREATE USER resultsEditor WITH PASSWORD '*';
```

В листинге 23 представлены SQL-команды, которые обеспечивают доступ новым пользователям к объектам базы данных. Первая команда предоставляет обоим пользователям право подключения к базе данных `cows_db`. Вторая команда предоставляет возможность обоим пользователям обращаться к объектам, расположенным в схеме public. Третья команда выдаёт пользователю `resultsEditor` право на использование всех последовательностей в схеме public, что необходимо для создания новых записей в таблицах, содержащих автоинкрементные поля (SERIAL).

Листинг 23: Общие права на базу данных

```
1 GRANT CONNECT ON DATABASE cows_db TO resultsReader,
    ↪ resultsEditor;
2 GRANT USAGE ON SCHEMA public TO resultsReader,
    ↪ resultsEditor;
3 GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO
    ↪ resultsEditor;
```

В листинге 24 представлены SQL-команды, с помощью которых новым пользователям были выданы права на чтение или изменение таблиц. Пользователю resultsReader выдано право только на чтение таблицы `Cow_lactation_diet_count`, а пользователю `resultsEditor` — на чтение и изменение таблиц `Cow`, `Lactation`, `Nutrition`.

Листинг 24: Права на таблицы

```
GRANT SELECT ON Cow_lactation_diet_count TO resultsReader;
GRANT SELECT, INSERT, DELETE, UPDATE ON Cow, Lactation, Nutrition TO resultsEditor;
```

В листинге 25 представлено изменение триггерных функций. Добавлено `SECURITY DEFINER` — выполнение функции с правами её создателя, а не вызывающего. Это необходимо для того, чтобы при изменении таблиц пользователем `resultsEditor` вызывались триггерные функции, несмотря на то что текущий пользователь не имеет доступа к таблице `Cow_lactation_diet_count`, обновляемой триггерами.

Листинг 25: Изменение триггерных функций

```
ALTER FUNCTION Cow_insert_trigger_function() SECURITY DEFINER;
ALTER FUNCTION Cow_delete_trigger_function() SECURITY DEFINER;
ALTER FUNCTION Nutrition_insert_trigger_function() SECURITY DEFINER;
ALTER FUNCTION Nutrition_delete_trigger_function() SECURITY DEFINER;
ALTER FUNCTION Nutrition_update_trigger_function() SECURITY DEFINER;
ALTER FUNCTION Lactation_insert_trigger_function() SECURITY DEFINER;
ALTER FUNCTION Lactation_delete_trigger_function() SECURITY DEFINER;
ALTER FUNCTION Lactation_update_trigger_function() SECURITY DEFINER;
```

4.2 Результаты работы

На рисунке 12 представлен вывод ошибки при попытке просмотра таблицы `Cow_lactation_diet_count` пользователем `resultsEditor`.

```
cows_db=> select * from Cow_lactation_diet_count limit 5;
ERROR:  permission denied for table cow_lactation_diet_count
```

Рис. 12: Ошибка доступа пользователя `resultsEditor` к таблице `Cow_lactation_diet_count`

На рисунке 13 представлен вывод строк из таблиц `Cow`, `Lactation`, `Nutrition` после запросов `SELECT` пользователем `resultsEditor`.

```
cows_db=> select * from cow limit 5;
select * from Lactation limit 5;
select * from Nutrition limit 5;
 id_cow | weight | age | id_breed
--------+--------+-----+----------
      1 |    671 |   6 |        1
      2 |    833 |   1 |        1
      3 |    502 |  13 |        1
      4 |    431 |  13 |        1
      5 |    759 |  11 |        1
(5 rows)

 id_lactation | number |   start    |    End     | id_cow | id_lactation_phase
--------------+--------+------------+------------+--------+--------------------
            1 |      1 | 2024-07-07 | 2024-08-05 |      1 |                  1
            2 |      1 | 2024-08-05 | 2024-11-16 |      1 |                  2
            3 |      1 | 2024-11-16 | 2024-12-23 |      1 |                  3
            4 |      1 | 2024-12-23 | 2025-01-26 |      1 |                  4
            5 |      1 | 2025-01-26 | 2025-02-19 |      1 |                  5
(5 rows)

 id_nutrition | id_cow | id_feeding_plan
--------------+--------+-----------------
            1 |      1 |              95
            2 |      1 |             286
            3 |      1 |             385
            4 |      1 |             317
            5 |      1 |             338
(5 rows)
```

Рис. 13: Успешный просмотр таблиц `Cow`, `Lactation`, `Nutrition` пользователем `resultsEditor`

На рисунке 14 представлен вывод строк из таблицы `Cow_lactation_diet_count` после запроса `SELECT` пользователем `resultsReader`.

```
cows_db=> select * from Cow_lactation_diet_count limit 5;
 id_cow | lactation_count | diet_count
--------+-----------------+------------
      1 |              30 |         16
      2 |              25 |         24
      3 |              30 |         16
      4 |              30 |         21
      5 |              35 |         18
(5 rows)
```

Рис. 14: Успешный просмотр таблицы `Cow_lactation_diet_count` пользователем `resultsReader`

На рисунке 15 представлен вывод ошибки при попытке просмотра таблицы `Cow` пользователем `resultsReader`.

```
cows_db=> select * from cow limit 5;
ERROR:  permission denied for table cow
```

Рис. 15: Ошибка доступа пользователя `resultsReader` к таблице `Cow`

На рисунке 16 представлен пример добавления записи в таблицу `Cow` пользователем `resultsEditor`, а также изменение и удаление этой записи.

```
cows_db=> insert into Cow (Weight, Age, id_Breed) values (132, 1, 1);
INSERT 0 1
cows_db=> SELECT * FROM Cow WHERE Weight=132 AND Age=1;
 id_cow | weight | age | id_breed
--------+--------+-----+----------
   6584 |    132 |   1 |        1
(1 row)

cows_db=> UPDATE Cow SET Age=2 WHERE id_Cow = 6584;
UPDATE 1
cows_db=> SELECT * FROM Cow WHERE Weight=132 AND Age=1;
 id_cow | weight | age | id_breed
--------+--------+-----+----------
(0 rows)

cows_db=> SELECT * FROM Cow WHERE Weight=132 AND Age=2;
 id_cow | weight | age | id_breed
--------+--------+-----+----------
   6584 |    132 |   2 |        1
(1 row)

cows_db=> DELETE FROM Cow WHERE id_Cow=6584;
DELETE 1
cows_db=> SELECT * FROM Cow WHERE Weight=132 AND Age=2;
 id_cow | weight | age | id_breed
--------+--------+-----+----------
(0 rows)
```

Рис. 16: Операции `INSERT`, `UPDATE` и `DELETE` в таблице `Cow` пользователем `resultsEditor`

На рисунке 17 представлен пример добавления записи в таблицу `Cow` пользователем `resultsEditor` и автоматическое добавление записи в таблицу `Cow_lactation_diet_count`.

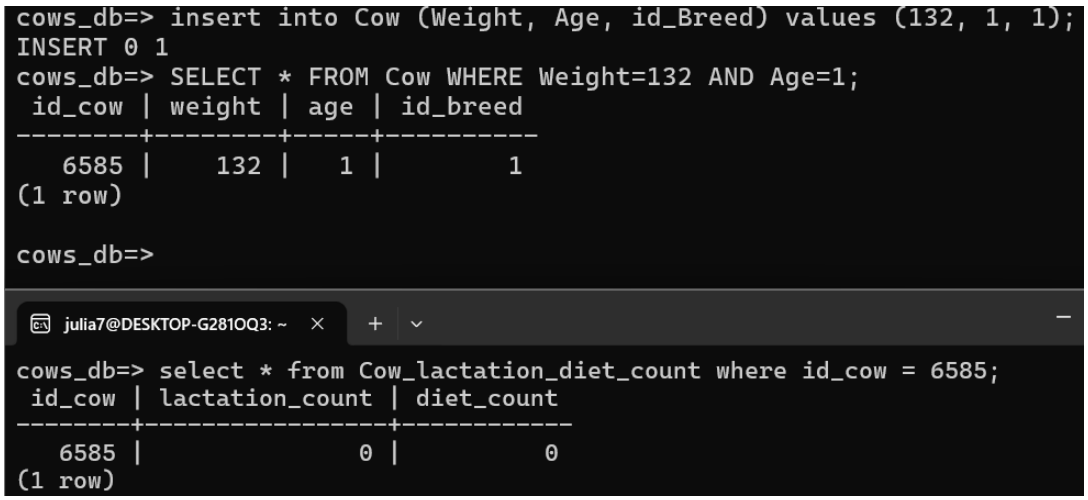


Рис. 17: Автоматическое обновление таблицы триггером при добавлении записи в таблицу `Cow`

В таблице 2 приведены сравнительные результаты проверки прав доступа пользователей.

Таблица 2: Результаты проверки прав доступа пользователей

№	**resultsReader**	**resultsEditor**
`SELECT * FROM Cow_lactation_diet_count LIMIT 5;`		
1	Результат запроса	Ошибка доступа
`SELECT * FROM Cow LIMIT 5;`		
2	Ошибка доступа	Результат запроса
`INSERT INTO Cow (Weight, Age, id_Breed) VALUES (160, 2, 3);`		
3	Ошибка доступа	Выполненный запрос
`UPDATE Cow SET Age = 2 WHERE id_Cow = 6584;`		
4	Ошибка доступа	Выполненный запрос
`DELETE FROM Cow WHERE id_Cow = 6584;`		
5	Ошибка доступа	Выполненный запрос
`SELECT * FROM Lactation LIMIT 5;`		
6	Ошибка доступа	Результат запроса
`SELECT * FROM Breed LIMIT 5;`		
7	Ошибка доступа	Ошибка доступа

Заключение раздела «Права доступа»

В результате выполнения работы были созданы две учётные записи пользователей. Были выданы базовые права на подключение к базе данных и использование схемы, а также права на использование последовательностей для пользователя `resultsEditor`. Затем были выданы права на таблицы: пользователю `resultsReader` — на чтение таблицы `Cow_lactation_diet_count`, а пользователю `resultsEditor` — на чтение и изменение таблиц `Cow`, `Lactation`, `Nutrition`. Для корректной работы триггеров при операциях от имени `resultsEditor` все триггерные функции были переведены в режим `SECURITY DEFINER`.

Список литературы

[1] Косарева Н.А., Чаунина Е.А., Ахмиева Г.Р., Петрова М.Ю. Влияние качества кормовых рационов на производство молока // Современное состояние, перспективы развития АПК и производства специализированных продуктов питания: материалы Международной научно-практической конференции, посвящённой юбилею Заслуженного работника высшей школы Российской Федерации, доктора технических наук, профессора Гавриловой Натальи Борисовны, 24 апреля 2020 года. – Омск: Омский ГАУ, 2020. – С. 700–706.